```json
{
"name": "DLQ_Retry",
"nodes": [
{
"parameters": {
"mode": "everyMinute",
"options": {}
},
"name": "Cron Trigger",
"type": "n8n-nodes-base.cron",
"typeVersion": 1,
"position": [0, 0],
"id": "dlq_retry_cron_001"
},
{
"parameters": {
"operation": "executeQuery",
"query": "SELECT * FROM v_booking_dlq_ready_for_retry LIMIT 10;"
},
"name": "Get Ready for Retry",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [250, 0],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "dlq_retry_get_002"
},
{
"parameters": {
"batchSize": 1,
"options": {}
},
"name": "Split In Batches",
"type": "n8n-nodes-base.splitInBatches",
"typeVersion": 3,
"position": [500, 0],
"id": "dlq_retry_split_003"
},
{
"parameters": {
"url": "https://n8n.stax.ink/webhook/booking-orchestrator",
"method": "POST",
"sendBody": true,
"bodyParameters": {
"parameters": [
{
"name": "provider_id",
"value": "={{ $json.provider_id }}"
},
{
"name": "service_id",
"value": "={{ $json.service_id }}"
},
```

```
{
"name": "start_time",
"value": "={{ $json.start_time }}"
},
{
"name": "customer_id",
"value": "={{ $json.customer_id }}"
},
{
"name": "chat_id",
"value": "={{ $json.chat_id }}"
}
]
},
"options": {
"timeout": 30000,
"retryOnFail": false
}
},
"name": "Retry Booking",
"type": "n8n-nodes-base.httpRequest",
"typeVersion": 4.4,
"position": [750, -100],
```

```
"id": "dlq_retry_booking_004"
},
{
"parameters": {
"conditions": {
"boolean": [
{
"value": true,
"operation": "equal",
"property": "={{ $json.success }}"
}
]
}
},
"name": "Retry Success?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [1000, -100],
"id": "dlq_retry_if_005"
},
{
"parameters": {
"operation": "executeQuery",
"query": "SELECT booking_dlq_resolve($1::bigint, 'auto_retry', 'Booking successfully retried
via DLQ_Retry');",
"options": {
"queryReplacement": "={{ $json.dlq_id }}"
}
},
"name": "Mark as Resolved",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [1250, -200],
"credentials": {
```

"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "dlq_retry_resolve_006"
},
{
"parameters": {
"operation": "executeQuery",
"query": "-- Update failure count and next retry time\nUPDATE booking_dlq\nSET failure_count
= failure_count + 1,\n   last_error_message = $1::text,\n   last_failed_at = NOW(),\n
next_retry_at = CASE \n   WHEN failure_count + 1 >= 5 THEN NULL\n   ELSE NOW() +
(POWER(2, failure_count + 1) || ' minutes')::INTERVAL\n   END\nWHERE dlq_id =
$2::bigint\nRETURNING dlq_id, failure_count;",
"options": {
"queryReplacement": "={{ $json.error_message || 'Retry failed' }}, {{ $json.dlq_id }}"
}
},
"name": "Update Failure Count",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [1250, 0],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "dlq_retry_update_007"
},
{
"parameters": {
"jsCode": "// Check if max retries reached (failure_count >= 5)\nconst item =
$input.first().json;\nconst maxRetries = 5;\n\nif (item.failure_count >= maxRetries) {\n //
Mark as discarded\n return [{\n   json: {\n   ...item,\n   should_discard: true,\n
message: `Max retries (${maxRetries}) reached. Moving to discarded.`\n   }\n }];\n} else
{\n // Continue with next retry\n return [{\n   json: {\n   ...item,\n
should_discard: false,\n   message: `Retry failed. Will retry again in ${Math.pow(2,
item.failure_count + 1)} minutes.`,\n   next_retry_at: new Date(Date.now() + Math.pow(2,
item.failure_count + 1) * 60000).toISOString()\n   }\n }];\n}",
"options": {}
},
"name": "Check Max Retries",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [1500, 0],
"id": "dlq_retry_check_max_008"
},
{
"parameters": {
"conditions": {
"boolean": [
{
"value": true,
"operation": "equal",
"property": "={{ $json.should_discard }}"
}

```json
]
}
},
"name": "Should Discard?",
"type": "n8n-nodes-base.if",
"typeVersion": 2.3,
"position": [1750, 0],
"id": "dlq_retry_if_discard_009"
},
{
"parameters": {
"operation": "executeQuery",
"query": "SELECT booking_dlq_discard($1::bigint, 'Max retries (5) reached after automatic
retry attempts');",
"options": {
"queryReplacement": "={{ $json.dlq_id }}"
}
},
"name": "Discard Item",
"type": "n8n-nodes-base.postgres",
"typeVersion": 2.6,
"position": [2000, -100],
"credentials": {
"postgres": {
"id": "SFNQsmuu4zirZAnP",
"name": "Postgres account"
}
},
"id": "dlq_retry_discard_010"
},
{
"parameters": {
"jsCode": "// Log retry result\nconst input = $input.first().json;\n\nreturn [{\n  json: {\n
dlq_id: input.dlq_id,\n    action: input.should_discard ? 'discarded' : 'retry_scheduled',\n
failure_count: input.failure_count,\n    message: input.message,\n    timestamp: new
Date().toISOString()\n  }\n}];",
"options": {}
},
"name": "Log Retry Result",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [2000, 100],
"id": "dlq_retry_log_011"
},
{
"parameters": {
"jsCode": "// No more items to process\nreturn [{\n  json: {\n    success: true,\n
message: 'DLQ retry batch completed',\n    processed_count: 0,\n    timestamp: new
Date().toISOString()\n  }\n}];",
"options": {}
},
"name": "No More Batches",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [750, 100],
"id": "dlq_retry_no_more_012"
},
```

```
{
"parameters": {
"jsCode": "// Standard Contract Output for retry completion\nconst result =
$input.all().pop()?.json || {};\n\nreturn [{\n  json: {\n    success: true,\n    error_code:
null,\n    error_message: null,\n    data: result,\n    _meta: {\n      source:
'DLQ_RETRY',\n      timestamp: new Date().toISOString(),\n      version: '1.0.0'\n    }\n
}\n}];",
"options": {}
},
"name": "Retry SCO",
"type": "n8n-nodes-base.code",
"typeVersion": 2,
"position": [2250, 0],
"id": "dlq_retry_sco_013"
}
],
"connections": {
"Cron Trigger": {
"main": [
[
{
"node": "Get Ready for Retry",
"type": "main",
"index": 0
}
]
]
},
"Get Ready for Retry": {
"main": [
[
{
"node": "Split In Batches",
```

```
"type": "main",
"index": 0
}
]
]
},
"Split In Batches": {
"main": [
[
{
"node": "Retry Booking",
"type": "main",
"index": 0
}
],
[
{
"node": "No More Batches",
"type": "main",
"index": 0
}
]
]
},
```

```
"Retry Booking": {
"main": [
[
{
"node": "Retry Success?",
"type": "main",
"index": 0
}
]
]
},
"Retry Success?": {
"main": [
[
{
"node": "Mark as Resolved",
"type": "main",
"index": 0
}
],
[
{
"node": "Update Failure Count",
"type": "main",
"index": 0
}
]
]
},
"Mark as Resolved": {
"main": [
[
{
"node": "Log Retry Result",
"type": "main",
"index": 0
}
]
]
},
"Update Failure Count": {
"main": [
[
{
"node": "Check Max Retries",
"type": "main",
"index": 0
}
]
]
},
"Check Max Retries": {
"main": [
[
{
"node": "Should Discard?",
```

```
"type": "main",
```

"index": 0
}
]
]
},
"Should Discard?": {
"main": [
[
{
"node": "Discard Item",
"type": "main",
"index": 0
}
],
[
{
"node": "Log Retry Result",
"type": "main",
"index": 0
}
]
]
},
"Discard Item": {
"main": [
[
{
"node": "Log Retry Result",
"type": "main",
"index": 0
}
]
]
},
"Log Retry Result": {
"main": [
[
{
"node": "Split In Batches",
"type": "main",
"index": 0
}
]
]
},
"No More Batches": {
"main": [
[
{
"node": "Retry SCO",
"type": "main",
"index": 0
}
]
]
},
"Retry SCO": {
"main": [

[
{
"node": "Split In Batches",
"type": "main",
"index": 0
}
]
]
}
},
"settings": {
"executionOrder": "v1"
},
"active": false,
"versionId": "dlq-retry-v1.0.0"
}